Technische Hochschule **Rosenheim**

Fakultät für Informatik

Wirtschaftsinformatik

Konzeption und Entwicklung eines KI-gestützten Chatbots zur automatischen Beantwortung von Supportanfragen unter Verwendung interner Datenquellen

Bachelor Thesis

von

Atakan Sentürk

Datum der Abgabe: tt.mm.jjjj
Erstprüfer: Prof. Dr. Marcel Tilly
Zweitprüfer: Prof. Dr. Gerd Beneken

Eigenständigkeitserklärung / Declaration of Originality

Hiermit bestätige ich, dass ich die vorliegende Arbeit selbständig verfasst und keine anderen als die angegebenen Hilfsmittel benutzt habe. Die Stellen der Arbeit, die dem Wortlaut oder dem Sinn nach anderen Werken (dazu zählen auch Internetquellen) entnommen sind, wurden unter Angabe der Quelle kenntlich gemacht.

I declare that I have authored this thesis independently, that I have not used other than the declared sources / resources, and that I have explicitly marked all material which has been quoted either literally or by content from the used sources.

Rosenheim, den tt.mm.jjjj

Vor- und Zuname

Kurzfassung

text

Schlagworte:

Inhaltsverzeichnis

1 Einführung **1**
- 1.1 Motivation . . . 1
- 1.2 Zielsetzung . . . 1

2 Grundlagen **2**
- 2.1 Techdivision & IT-Services . . . 2
- 2.2 Chatbots . . . 2
  - 2.2.1 ELIZA . . . 2
  - 2.2.2 Parry . . . 3
  - 2.2.3 ALICE . . . 4
  - 2.2.4 Siri, Google Assistant, Alexa . . . 4
  - 2.2.5 ChatGPT, Claude, Gemini . . . 5
- 2.3 Künstliche Intelligenz . . . 5
  - 2.3.1 Schwache künstliche Intelligenz . . . 6
  - 2.3.2 Starke künstliche Intelligenz . . . 6
  - 2.3.3 Natural Language Processing . . . 7
  - 2.3.4 Neuronale Netzwerke . . . 7
  - 2.3.5 Training . . . 9
  - 2.3.6 Encoder-Decoder Architektur . . . 9
  - 2.3.7 Attention Mechanismus . . . 10
- 2.4 LLMs . . . 10
  - 2.4.1 Transformer . . . 11
  - 2.4.2 Tokenizer . . . 12
  - 2.4.3 Pretraining . . . 13
  - 2.4.4 Finetuning . . . 13
  - 2.4.5 Inference & Generation . . . 14
  - 2.4.6 Limitationen & Halluzinationen . . . 14
  - 2.4.7 Prompting . . . 15
- 2.5 Vektor Datenbanken . . . 16
- 2.6 Embedding Modelle . . . 16
- 2.7 RAG . . . 17
  - 2.7.1 Naive RAG . . . 17
  - 2.7.2 Advanced RAG . . . 18
- 2.8 llama.cpp . . . 18
- 2.9 Ollama . . . 19
- 2.10 Ragas . . . 20
- 2.11 AI Frameworks . . . 20
  - 2.11.1 Haystack . . . 20
  - 2.11.2 Langchain . . . 20
  - 2.11.3 LlamaIndex . . . 21
- 2.12 Huggingface / MTEB . . . 21

2.13 Google . . . 22
2.13.1 Google Cloud . . . 22
2.13.2 Google Chat . . . 22

3 Anforderungen **23**

4 Konzeption **24**
4.1 Auswahl der Chatmodelle . . . 24
4.2 Auswahl der Embedding Modelle . . . 25
4.3 Prompting . . . 25
4.4 Auswahl der Vektor Datenbank . . . 25
4.5 Server . . . 26
4.6 Chainlit . . . 26
4.7 Verbindung zu Google Chat . . . 26
4.8 Architektur des Systems . . . 26
4.8.1 Verbindungen . . . 26
4.8.2 Chatbot . . . 26
4.8.3 Datenupdates . . . 27
4.8.4 Server . . . 28
4.8.5 Anbindung Google Chat . . . 28

5 Entwicklung **29**
5.1 Vorbereitung . . . 29
5.1.1 Modelle . . . 29
5.1.2 Google Cloud . . . 29
5.1.3 Python Umgebung . . . 29
5.2 System . . . 29

6 Vergleich **30**
6.1 Evaluierung . . . 30
6.2 RAGAS Vergleich . . . 30
6.2.1 Testset . . . 30
6.3 Umfrage Vergleich . . . 31
6.3.1 Fragebogen . . . 31

A Erstes Kapitel des Anhangs **32**

Abbildungsverzeichnis

Tabellenverzeichnis

Abkürzungen

LLM Large Language Model

NLP Natural Language Processing

1 Einführung

1.1 Motivation

Die Techdivision ist intern in sogenannten Services strukturiert, die häufig ein intensives Zusammenarbeiten erfordern. Diese Zusammenarbeit findet über verschiedene Kommunikationskanäle statt, darunter JIRA Boards und Support-Chats. Oft werden Routinefragen gestellt, die automatisiert werden könnten.

Durch den Aufstieg von verschiedenen textbasierten Chat-Modellen wie OpenAIs ChatGPT oder Googles Gemini, hat man in den letzten Jahren gesehen, zu was Modelle inzwischen in der Lage sind. Sie sind nicht mehr nur in der Lage, mit Menschen zu kommunizieren, sondern helfen diesen auch bei ihren Jobs oder Schulangelegenheiten. Verschiedene trainierte Modelle sind inzwischen in der Lage, komplizierte Probleme zu lösen und Menschen das Leben einfacher zu machen.

Dadurch stellt sich speziell in den Support-Chats die Frage, wie viele der Supportanfragen sich durch einen KI-gesteuerten Chatbot ohne Hilfe eines Mitarbeiters lösen lassen würden und inwiefern ein Chatbot zur Entlastung der Teams und zur schnelleren Lösung von Problemen beitragen könnte. Durch die Automatisierung einfacher Supportanfragen könnte zum einen Zeit gespart, zum Anderen auch die Ressourcen der Teams effektiver genutzt werden. Mitarbeiter könnten sich verstärkt auf die schwierigeren Supportanfragen konzentrieren, während der Chatbot die repetitiven und einfachen klärt, was nicht nur das Team entlastet, sondern auch die Reaktionszeit auf Supportanfragen erhöht.

1.2 Zielsetzung

Das Ziel der Arbeit besteht darin, einen Chatbot zu konzipieren und einen Prototyp zu entwickeln, der mit Hilfe der Daten der internen Datenquellen der Techdivision Support leistet und Routinefragen beantwortet. Der Chatbot soll idealerweise innerhalb von Google Chat integriert werden. Dabei ist es wichtig, dass der Chatbot, statt in Einzelchats, in einer Chat Gruppe läuft. Diese beinhaltet neben den Support Suchenden, ebenfalls Mitarbeiter des jeweiligen Services. Auf diese Weise kann, falls notwendig, extern Eingegriffen werden. Außerdem sollte der Chatbot in der Lage sein zu überprüfen, wie viele der aktuellen Supportanfragen er eigenständig beantworten oder lösen kann. Falls der Chatbot keine Antwort auf eine Anfrage haben sollte, soll er in den angebundenen Datenquellen nachschlagen können, um mit dem Wissen seine Wissensbasis zu erweitern und die Fragen entsprechend zu beantworten.

2 Grundlagen

2.1 Techdivision & IT-Services

TD kurz vorstellen Die Techdivision ist ein Unternehmen, welches 2007 in Kolbermoor, Deutschland gegründet wurde. Der Grundstein für die Firma wurde aber bereits 1997 in der Garage von Tim Wagner, dem aktuellen CTO, gelegt. Zusammen mit Thomas Kalteis begann hier eine kleine Technologie-Schmiede. Im Jahr 2007 wurde daraus eine GmbH, gegründet von den Brüdern Stefan und Josef Willkommer.

2.2 Chatbots

Laut Wikipedia ist ein Chatbot "ein Chatbot oder kurz Bot ein textbasiertes Dialogsystem, das Chatten mit einem technischen System erlaubt" [**?**]. Es gibt verschiedene Arten von Chatbots, diese unterscheiden sich aber in der Art und Weise, wie sie auf die Eingaben des Nutzers antworten. Chatbots, wie OpenAis ChatGPT und Anthropics Claude, haben einen Bereich zur Texteingabe und einen Bereich, in dem sie ihre Ausgabe darstellen können. Mit Chatbots wie diesen kann man heutzutage über die natürliche menschliche Sprache kommunizieren und sie versuchen auf dieselbe Art und Weise zu antworten.

Die Generation des Textes eines Chatbots kann in drei verschiedene Kategorien untergeordnet werden: abfragebasiert, generationsbasiert und regelbasiert [**?**]. Abfragebasierte Chatbots haben eine Wissensdatenbank in der sie nach dem am besten zur Eingabe passenden Eintrag suchen. Generationsbasierte Chatbots generieren ihre Antworten selbst, wie ein paar dieser Modelle ihre Antworten generieren wird in **??** erklärt. Regelbasierte Chatbos antworten nach vorher in ihrem Skript bestimmten Regeln. Hierbei vergleichen sie die Eingabe mit ihren vorher definierten Regeln und transformieren die Eingabe, falls möglich, in eine vorher definierte Satzstruktur. [**?**] Im nächsten Schritt wird darauf eingegangen, wie sich Chatbots in dieser Hinsicht mit der Zeit verändert haben.

Im Jahr 1950 hat Alan Turing den Turing Test vorgeschlagen, bei dem es darum geht zu erkennen, ob eine Maschine das gleiche Denkvermögen hat wie ein Mensch. Bei diesem Test werden einem Tester zwei Gesprächspartner vorgelegt, von denen einer ein Mensch und der andere eine Maschine ist. Der Tester führt dann ein Gespräch mit beiden Gesprächspartnern und befragt diese. Nach der Befragung wird überprüft, ob der Tester in der Lage ist, zu sagen, welcher der Partner eine Maschine ist. Falls der Tester nicht in der Lage ist zu erkennen, welcher der Partner die Maschine ist, gilt der Turing Test als bestanden und die Maschine ist laut Turing intelligent. [**?**]

2.2.1 ELIZA

Die Geschichte der Chatbots beginnt in den 1960er Jahren mit dem ersten Chatbot aus dem Jahr 1966. Damals waren die Chatbots regel basiert. Der Name des Chatbots war ELIZA und er wurde von Joseph Weizenbaum am MIT entwickelt. Mit der Regel basierung von ELIZA

hat Weizenbaum fünf Grundprobleme aufgestellt, welche er für die Entwicklung von ELIZA bekämpfen musste . Die Probleme waren:

1. Die Identifikation von Stichwörtern
2. Das Finden des minimalen Kontexts
3. Die Auswahl der geeigneten Transformation
4. Generation von Antworten bei Abwesenheit von Stichworten
5. Bearbeitung der Eliza Skripte

[**?**].

ELIZA wurde so entwickelt, dass sie versucht hat, menschliche Sprache zu imitieren, indem sie den eingegebenen Text nach Stichwörtern durchsucht. Stichwörter sind Wörter, denen besonders hohe Wichtigkeit laut dem ELIZA-Script gegeben wurde. Es lässt sich vermuten, dass hierbei besonders Wörter mit einem hohen Informationsgehalt verwendet wurden. Jedes Stichwort hatte noch einen sogenannten RANK, welcher die Wichtigkeit des jeweiligen Stichworts im Vergleich zu anderen Stichworten festgelegt hat. Nachdem die Stichwörter entdeckt wurden, betrachtet ELIZA die "Transformation Rules", die in ihrem Skript bestimmt wurden. Diese bestehen aus einer Zerlegungs und einer Zusammensetzungs-Phase. Es gibt verschiedene Regeln für beide Phasen. Anhand der Syntax des Satzes wird ausgemacht, welche Regel für die Zerlegung in Frage kommt und anhand dieser wird dann die Zusammensetzung ausgemacht. Dieser Zusammengesetzte Satz wird im Anschluss ausgegeben Falls jedoch kein Stichwort gefunden wurde, gibt ELIZA entweder eine inhaltslose Antwort oder eine frühere Antwort zurück. Eliza verhält sich wie ein Psychotherapeut. Das heißt, der Nutzer beginnt das Gespräch und Eliza greift Punkte hauptsächlich aus den Eingaben des Nutzers auf, ohne großen Zusammenhang mit Themen außerhalb dieses Gesprächs. Der Bot kennt keinen Kontext sondern antwortet, je nach seinem aktuellen Skript auf den aktuellen Satz. Selbst Weizenbaum selbst hat scheinbar mit der Eröffnung in seine Arbeit gezeigt, dass er selbst nicht der Meinung war, dass ELIZA wirklich ein intelligentes System ist.

Er ist der Meinung, dass Programme im Bereich der künstlichen Intelligenz und im Bereich des heuristischen Programmierens so aufgebaut sind, dass sie auf wundersame Weise funktionieren, sodass sie selbst erfahrene Beobachter faszinieren. Aber sobald man herausfindet, wie diese Programme funktionieren, verlieren sie ihre Magie und es entarnt sich als nichts mehr als eine Reihe von Anweisung, die alleine betrachtet sehr simple sind. Das Ziel seiner Forschungsarbeit ist es, die Magie hinter Eliza zu enttarnen, da er der Meinung ist, dass es wenig andere Dinge gibt, die diese Enttarnung so sehr nötig hatten, wie Eliza [**?**].

2.2.2 Parry

Nach Eliza kam ein Chatbot namens Parry der an der Stanford University entwickelt wurde. Während Eliza eher einen Psychotherapeuten imitiert hat, hat Parry einen Patienten mit paranoider Schizophrenie imitiert. Dieser Bot war im Jahr 1979 Teil eines Experiments, der einem Turing Test gleicht. Hier wurden verschieden Psychotherapeuten vor Bildschirme gesetzt und mussten erraten, ob die gegenüber gestellten Aussagen von einem echten Schizophrenen Patienten kommen oder von einem Chatbot. Teil des Experiments waren 5

Psychiater, die jeweils 2 "Patientenßur Beurteilung bekommen haben. Der erste lag bei beiden richtig, der zweite lag bei beiden falsch. Der dritte dachte beide Patienten sind echte Patienten und die letzten zwei dachten beide sind Chatbots. Das Problem hierbei war, dass der Pool an Probanden mit 5 Psychiatern sehr klein war. Dazu kommt noch der Fakt, dass Patienten mit Schizophrenie ein gewisses Level an Unverständlichkeit in ihren Konversationen haben, welches die Beurteilung, ob es generell ein guter Chatbot ist erschwert. Heute wird Parry eher als ein schwacher Chatbot gesehen, da er nicht von Konversationen lernen kann, keinen richtigen Kontext hat und sehr langsam ist. Natürlich ist ein Teil davon abhängig von den technischen Limitationen die es damals gab, aber es ist dennoch wichtig zu beachten [**?**].

2.2.3 ALICE

1995 wurde der Internet Chatbot ALICE veröffentlicht. Er war im Internet zu finden und er wurde als Open Source Software veröffentlicht, hat aber nicht viele Mitentwickler gefunden, bis die Software im Jahre 1998 von der Programmiersprache SETL auf die Programmiersprache Java gewechselt ist. Das Problem war, dass SETL eine eher unbekannte Sprache war. Im Laufe der Jahre wurde das Projekt mehrmals mit verschiedenen Preise ausgezeichnet, wie beispielsweise dem Loebner Prize. Dieser Bot wurde von Weizenbaums Eliza inspiriert und war selbst die Inspiration für den Film "Her", in der die Beziehung zwischen Menschen und Maschinen beleuchtet wird [**?**]. Alice hatte eine viel größere Sammlung an Keywords, auf die sie reagieren konnte als Eliza, aber von den Grundprinzipien her, hat sie auf die selbe Art und Weise Keywords in den Eingaben des Nutzers gesucht und dann je nach ihren Datenbankeinträgen darauf geantwortet [**?**, **?**].

2.2.4 Siri, Google Assistant, Alexa

Nach diesen anfänglichen Versuchen im Bereich der Chatbots, welche vorallem im Bereich der Psychologie aufgebaut wurden und die regelbasiert waren wurden immer weiter Fortschritte gemacht.

Eine große Neuerung im Jahr 2010 war Siri, ein persönlicher Assistent, auf dem eigenen Handy. Siri, welches nur auf Apple Geräten zur Vefügung steht, hilft den Nutzern, welche über Text oder Sprachbefehle mit dem Assistenten kommunizieren können, mit verschiedenen Aufgaben. Unter anderem können die Nutzer mittels Siri unter anderem Erinnerungen setzen, Nachrichten senden und Anrufe tätigen. [**?**]

Sprachassistenten wie Siri gibt es auch auf anderen Plattformen, wie auf Samsung Galaxy Geräten mit Galaxy Ai [**?**] und verschiedenen Android Geräten mit Google Assistant [**?**]. Diese Bots haben ähnliche Funktionen wie Siri und versuchen dem Nutzer zu helfen[**?**].

SmartHome Geräte, wie Smarte Lampen und Steckdosen kann man mit hilfe von diesen Sprachassisten auch kontrollieren.

Man kann auf manche Sprachassistenten auch teilweise über extra Geräte, die für sie hergestellt wurden zugreifen. Ein Beispiel hierfür Amazon Alexa, die der Sprachassistent von Amazon ist, auf dem Echo Dot. Der Echo Dot ist ein kleiner Lautsprecher, welcher mit dem internet verbunden ist. Durch diesen kann man dann mit Alexa kommunizieren. Diese Sprachassisten werden immer weiter entwickelt und bekommen dadurch neue Funktionen, mehr Sprachen oder ein besseres Verständnis für die menschliche Sprache.

2.2.5 ChatGPT, Claude, Gemini

Mit der erhöhten Popularität von Chatbots wie ChatGPT und Claude sind Chatbots, wie diese, die größte Neuerung in dieser Welt. Diese Chatbots basieren auf Large Language Models, welche in 2.4 genauer erklärt werden. Nutzer können teils über Apps und teils über die Website, des jeweiligen Services, mit diesen Bots kommunizieren.

Den bots kann man fragen stellen und diese werden vom jeweiligen System beantwortet. Sie können nicht nur per Text antworten, sondern auch in Form von Bildern. ChatGPT kann durch die DALL·E Integration, welches ein weiteres System von OpenAI ist, Bilder generieren. ChatGPT hat in der Premium Variante die Möglichkeit Bilder zu generieren. Die Bots haben die Fähigkeit viele verschiedene Dinge zu tun. Unter anderem können sie Code, einen Essay oder ein Film Script schreiben.

2.3 Künstliche Intelligenz

Künstliche Intelligenz ist ein Teil der Informatik der sich damit beschäftigt, Aufgaben, die früher menschliche Intelligenz benötigt hat, von Maschinen bzw. Computern übernehmen zu lassen. Zu den Aufgaben gehören unter anderem:

- Erkennen von Mustern
- Treffen von Entscheidungen
- Verstehen von Sprache

Durch Fortschritte in der künstlichen Intelligenz konnten Forstschritte in all diesen Aufgabenbereichen erreicht werden. Die verschiedenen Aufgaben werden teils in verschiedenen Teilbereichen der künstlichen Intelligenz behandelt. Ein paar der Teilbereiche sind:

- Computer Vision
- Natural Language Processing
- Robotik

Nun ist die Frage wie Intelligenz definiert werden kann. Das ist keine einfache Aufgabe und nicht der Schwerpunkt dieser Arbeit. Im Kontext dieser Arbeit wird das Buch “Knowledge Science – Grundlagen: Methoden der Künstlichen Intelligenz für die Wissensextraktion aus Texten” von Lanquillong und Schacht betrachtet. Laut Lanquillon und Schacht ist eine Möglichkeit ein paar Eigenschaften anzuschauen, welche an menschliches Verhalten angelehnt sind anzuschauen und davon ausgehend künstliche Intelligenz betrachten oder definieren. Diese Liste ist offen und nicht vollständig. Die Eigenschaften die im Buch genannt werden sind:

- Wahrnehmung
- Schlussfolgerung
- Handeln
- Planung und zielgerichtete Problemlösung
- Kommunikation

- Anpassungsfähigkeit und Lernen
- Autonomie
- Kreativität
- Reflexion und Bewusstheit
- Ästhetik
- Organisation

Die meisten Menschen weisen alle diese Eigenschaften auf, aber es gibt noch keine künstliche Intelligenz, die all diese Eigenschaften aufweisen kann. Da es noch kein System gibt, welches in diese Kategorie fällt, aber es bereits Systeme gibt, die teils ein paar dieser Eigenschaften aufweisen und dazu noch Probleme in eng definierten Problembereichen lösen können. Ist eine weitere Unterscheidung innerhalb des Bereichs der künstlichen Intelligenz nötig. Daraus formen sich die zwei Begriffe schwache künstliche Intelligenz und starke künstliche Intelligenz [**?**].

2.3.1 Schwache künstliche Intelligenz

Die schwache künstliche Intelligenz ist ein System, welches spezifische Probleme innerhalb einer bestimmten Domäne lösen kann. Diese Art der künstlichen Intelligenz wird darauf trainiert, Probleme innerhalb eines eng begrenzten Bereichs zu lösen, indem sie speziell auf diesen Bereich trainiert wird und im Laufe dessen dann die Regeln dieses Bereichs kennen lernt. Nachdem es trainiert wurde, kann die schwache künstliche Intelligenz durch die beim Training erlernten Regeln und Muster, Probleme innerhalb der Domäne in der es trainiert wurde lösen. Es kann dieses "Wissenäber nicht auf andere Problembereiche übertragen und dadurch Probleme in dieser neuen Domäne lösen.

Ein Beispiel für eine schwache künstliche Intelligenz wäre ein System, welches darauf trainiert wurde Texte von einer Sprache in einer andere zu übersetzen. Das System kann vielleicht innerhalb dieser Domäne die Aufgabe sehr gut erfüllen, was heißt, dass sie die Texte mit hoher Genauigkeit in die Zielsprache übersetzen kann, ohne den Verlust von Informationen, aber egal wie gut es in dieser Domäne ist, bleibt das System auf diese Domäne beschränkt. Das heißt, es kann das erlente Wissen, nicht auf unverwandte Aufgaben übertragen. [**?**].

2.3.2 Starke künstliche Intelligenz

Starke künstliche Intelligenz oder Allgemeine künstliche Intelligenz, wäre ein System, welches nicht nur die in 2.3 genannte Liste an Eigenschaften aufweisen könnte, sondern auch in der Lage wäre, Probleme zu lösen, ohne explizit für diese trainiert worden zu sein. Die starke künstliche Intelligenz, wäre in der Lage ihre erlentes Wissen zu verallgemeinern und es dadurch, ähnlich wie ein Mensch, auf neue Anwendungsbereiche anzuwenden. Darüber hinaus, wäre eine starke künstliche Intelligenz ebenfalls in der Lage sich selbst weiterzuentwickeln und zu lernen, ohne das menschliches Eingreifen erforderlich wäre. Diese Fähigkeit würde bedeuten, dass starke künstliche Intelligenz, durch diese Selbstverberbesserungsfähikeit in der Lage wäre ihr eigenes System und ihre eigenene Algorithmen zu optimieren und somit irgendwann die menschliche Intelligenz zu übertreffen [**?**].

Solch ein System würde nicht nur durch seine Problemlösungsfähigkeit ausgezeichnet sein, sonder ebenfalls durch Dinge wie kreatives Denken und emotionale Intelligenz. Dadurch wäre sie in fast allen Bereichen einsetzbar, in denen diese Eigenschaften gefragt sind [**?**].

Mit dem Aufstieg von künstlichen Intelligenzen im Auge der Öffentlichkeit, zeigen sich einige Menschen besorgt, was passieren würde, wenn solch ein System erschaffen werden würde. Es ist jedoch zu betonen, dass ein solches System noch nicht existiert und es ist davon auszugehen, dass dies auch in naher Zukunft nicht der Fall sein wird [**?**].

2.3.3 Natural Language Processing

Natural Language Processing oder natürliche Spracherkennung, ist ein Teil des Machine Learnings der sich auf die Arbeit mit menschlicher, natürlicher Sprache fokussiert. Das Ziel von Natural Language Processing (NLP) ist es, Computern die Möglichkeit zu geben, menschliche Sprache zu verstehen und zu verarbeiten, um mit den Menschen kommunizieren zu können. Die schließt Kommunikation in Form von Audio als auch über Text mit ein.

Natural Language Processing (NLP) ermöglicht es Computern, durch Kombination verschiedener Machine Learning Verfahren und andere linguistischen Verfahren menschliche Sprache zu verstehen und diese auch wiederzugeben. Diese System können im Zusammenhang damit, Texte zusammenzufassen, sie zu analysieren, ihre Bedeutung zu verstehen und mit Hilfe dieser Fähigkeiten dann sinnvolle und passende Antworten zu generieren.

Anwendungen die Natural Language Processing (NLP) nutzen sind vielfältig und beispielsweise Chatbos und Übersetzungssoftware implementieren auch NLP Verfahren.[**?**]

Da Sprache oft kontextabhängig ist und manche Sprachen, wie Japanisch komplette Satzteile weg lassen, wenn diese durch den Kontext erkenntlich sein sollten, ist es ein wichtiger Teil des Natural Language Processing (NLP), in der Lage zu sein kontextuelle Informationen zu berücksichtigen. Dadurch kann das System Kontext verstehen und richtig interpretieren. Dies wird über verschiedene Methoden erreicht, wie Tokenisierung, Parsing und Named Entity Recognition. Tokenisierung ist ein Verfahren, welches LLMs nutzen. Dieses wird in 2.4.2 genauer erklärt.[**?**]

NLP stellt eine wichtige Schlüsselstelle in der Human-Computer Interaction oder zu deutsch Mensch-Computer-Interaktion dar. Sie hilft nicht nur dabei menschliche Kommunikation mit Computern einfacher und schneller zu machen, sondern gibt auch Menschen, die beispielweise aufgrund von körperlichen Problemen ein Handy nicht ohne Probleme bedienen können, die Möglichkeit mit diesen Geräten zu interagieren[**?**].

2.3.4 Neuronale Netzwerke

Neuronale Netze sind ein Teil des Maschinellen Lernens, welche versucht sich stark an dem Aufbau eines biologischen Gehirns zu orientieren. Dementsprechend bestehen sie aus Tausenden oder sogar Millionen “Neuronen”, welche durch Knotenpunkte dargestellt werden. Diese “Neuronen” sind stark miteinander verbunden. Sie sind über verschiedene Reihen miteinander verbunden, wie in der Abbildung 2.1 dargestellt.

Jeder Eingang an einem Neuron hat eine Gewichtung. Wenn die Gewichtung eines Eingangs 0 ist, dann heißt es, dass zwei Knoten keine Verbindung miteinander haben. Wenn die Gewichtung positiv ist, dann hat es eine bestärkende Wirkung und, falls die Gewichtung negativ ist, dann hat es eine hemmende Wirkung.

Jedes Neuron hat zudem einen sogenannten Bias, der Bias wird benötigt, damit man die Aktivierungsschwelle der Aktivierungsfunktion verschieben kann. Dies sorgt für eine

erhöhte Flexibilität, da der Wert der Aktivierungsfunktion ohne den Bias immer null wäre, wenn alle Eingaben null sind. Durch den Bias hat das Neuron also eine Grundaktivität, welche während dem Training angepasst werden kann, damit die Ergebnisse besser für den eigenen Fall werden. Das aktuelle Neuron summiert dann alle gewichteten Werte und addiert den Bias. Wie in dieser Formel dargestellt:

$$\sum_i w_i x_i + b$$

Der summierte Wert wird durch die Aktivierungsfunktion transformiert und das Ergebnis wird dann dem nächsten Neuron in der nächsten Schicht weitergeleitet. Typische Aktivierungsfunktionen sind

- Sigmoid Function :
$$f(x) = \frac{1}{1 + e^{-x}}$$
- ReLu (Rectified Linear Unit):
$$f(x) = \max(0, x)$$
- Tanh:
$$f(x) = tanh(x)$$

[**?**] Die zwei Schichten die es immer gibt sind die Eingabe Schicht und die Ausgabe Schicht. Dazwischen kann es verstecke Schicht geben, die benötigt werden um komplexere Zusammenhänge zu bearbeiten, muss es aber nicht. Die Eingabe Schicht erhält die Eingabedaten von außen und gibt diese dann weiter an die nächste Schicht. Als nächstes werden die Daten weiter an eine verborgene Schicht gegeben, die die Werte dann bearbeitet. Es gibt X verschieden verborgene Reih in einem Neuronalen Netzwerk. Die Anzahl der Reihen und die Anzahl der Neuronen pro versteckter Schicht kann sich unterscheiden. Es gibt hierbei keine allgemeingültige Größe, die für alle Fälle passt. Stattdessen kommen hierbei Erfahrung und Experimentation ins Spiel.

Es kann sogar sein, dass gar keine versteckte Reihen notwendig sind, wenn das Model linear seperierbar ist. Das liegt daran, dass ein Modell ohne versteckte Schichten bereits in der Lage ist diese Grenze zu erlernen und die Daten dann korrekt zu klassifizieren.

Die Anzahl der Neuronen schwankt zwischen verschiedenen Neuronalen Netzen und auch innerhalb eines Neuronalen Netzes schwankt die Anzahl der Neuronen zwischen den verschiedenen Schichten. Bei der Eingabe Schicht kommt es auf die Trainingsdaten an, diese legen die Anzahl der Neuronen fest. Um genau zu sein, kommt es auf die Anzahl der Features an, die die Trainingsdaten beinhalten. Bei einem Neuronalen Netz, welches mit Daten aus einer normalen Tabelle trainiert wird, ist das beispielsweise die Anzahl der Spalten, bei einem bei einem Neuronalen Netz, welches mit Bildern trainiert werden soll, könnten das die einzelnen Pixel des jeweiligen Bildes sein.

[**?**] Die Anzahl der Neuronen in der Ausgabe Schicht kommt darauf an, welche Art von Modell man betrachtet. Es wird hier vor allem zwischen zwei verschiedenen Arten unterschieden. Die zwei Arten die es gibt sind Regressoren und Klassifikatoren. Bei Regressoren gibt es ein Ausgabe Neuron. Bei Klassifikatoren kann es mehrere geben.

Regressoren ordnen die Ergebnisse im Regelfall in eine von zwei Optionen ein. Der Wert den das Modell berechnet ist ein Wert zwischen 0 und 1. Aber die Ausgabe des Neuronalen Netzes ist entweder eins oder null. Hierbei gibt es einen Schwellenwert, der, wenn er

überschritten wird, die Ausgabe des Modells auf eins bringt und, wenn dieser Wert nicht überschritten wird, ist die Ausgabe null. Ein Regressor klassifiziert die Daten in eine von zwei Gruppen, wenn der Wert null ist, gehört die Eingabe in eine Gruppe und bei eins in die jeweils andere.

Beispiele hierfür sind:

- Gesichtserkennung
- Stimmerkennung
- Spamfilter in emails

Die Klassifikatoren haben ein oder auch mehrere Ausgabe Neuronen. Die Anzahl der Neuronen kommt auf die Anzahl der möglichen Ausgaben an. Beispielsweise könnte man ein System betrachten, welches lernen soll, welche Art von Fahrzeug auf einem Bild dargestellt ist. In der Abbildung 2.2 ist ein simpler Decision Tree dargestellt. Anhand dieser kann man sehen, welche Optionen es als Asugaben gibt. In dieser Situation hätte die Ausgangs Schicht drei Neuronen, da es drei mögliche Ausgaben gibt.

2.3.5 Training

2.3.6 Encoder-Decoder Architektur

Bevor auf die Transformer Architektur eingegangen werden kann, ist es wichtig, die vorangegangenen Architekturen zu verstehen. Dies ist wichtig, um zu verstehen, wie es zu den heutigen Architekturen gekommen ist und um zu erkennen, welche Vorteile diese bieten. Diese Technologien haben sich damals vorallem im Kontext des Sequence-to-Sequence Learnings weiterentwickelt. Sequence-to-Sequence Learning oder kurz Seq2Seq Learning, bringt eine Sequenz variabler Länge in eine andere Sequenz variabler Länge.[**?**] Beispiele hierfür wären Text Zusammenfassungen und Übersetzungen. Die Architektur aus der sich der Transformer, der in dem Abschnitt **??** behandelt wird, der in den LLMs heutzutage verwendet wird, entwickelt hat, ist die Encoder - Decoder Architektur.

Die Encoder-Decoder Architektur besteht aus zwei Rekurrenten Neuralen Netzen. Wie ein Neurales Netz funktioniert wurde bereits im Abschnitt 2.3.4 behandelt. Das besondere an einem Rekurrenten Neuralen Netz ist, dass dieses die vorherigen Aktivierungen der einzelnen Neuronen mit berücksichtigt. Dadurch, dass die vorangeganen Aktivierungen ebenfalls beachtet werden, können Kontexte erkannt werden und Sätze können sinnvoll übersetzt werden.

Hierbei erhählt der Encoder eine Eingabe Sequenz, welche dann Kontext Vektoren erstellt, welcher vom Decoder in eine Ausgabe Sequez umgewandelt wird. Das revolutionäre an dieser Technologie war damals, dass die Eingbae und Ausgabesequenzen variable Längen haben konnten. Der Kontext Vektor hatte jedoch eine feste Länge. Dieses Originale Encoder-Decoder Modell hatte trotz der Vorteile, die es geboten hat, jedoch einige Probleme:

1. Sequenzielles Verarbeiten: RNNs und LSTMs verarbeiten Sequenzen sequentiell. Dadurch, dass die Daten nich parallel verarbeitet werden, kommt es zu längeren Trainingszeiten.
2. Abhängigkeiten: In RNNs ist es schwierig langfristige Abhängigkeiten zu erfassen, aufgrund des Vanishing Gradient Problems, welches im Abschnitt 2.4.7 genauer erklärt wird.

3. Festes Kontextfenster: Da die meisten RNNs und LSTMs eine feste Kontextgröße haben, kann nur eine begrenzte Menge an Informationen als Kontext, zu einem bestimmte Zeipunkt, beachtet werden. Dadurch kann es zum Verlust von Informationen oder zur Verringerung der Genauigkeit der Antworten kommen. Besonders bei Sätzen die länger sind als die, in den Trainingsdaten, kann es hier zu Problemen kommen.

[**?**, **?**]

2.3.7 Attention Mechanismus

Die Idee auf die die Transformer aufbauen ist die des Attention Mechanism, welcher 2014 in der Arbeit “Neural Machine Translation by Jointly Learning to Align and Translate” von Bahdanau et al. vorgestellt wurde. Zu Beginn wurde der Mechanismus genutzt um die in 2.3.6 beschriebene Encode-Decoder Architektur zu verbessern, indem es vorallem auf das Problem der festen Vektorgröße des Kontextfensters eingeht.

Die Grundidee hierbei ist, dass jedes mal, wenn das System ein Wort übersetzt, es im zu übersetzenden Satz an den Stellen an denen sich die relevantesten Informationen sucht und mithilfe der Kontextvektoren der Wörter an diesen Stellen und der Kontextvektoren der Wörter die bereits übersetzt wurden, wird das nächste Wort generiert.

Der größte Unterschied zu originalen Encoder-Decoder Architektur ist dadurch, dass diese neue Architektur nicht versucht die gesamte Eingabesequenz in einen Kontextvektor umzuwandeln, sondern die Eingabesequenz in eine Sequenz von Vektoren umzuwandelt. Dadurch können die Modelle besser mit längeren Sätzen umgehen.

[**?**]

2.4 LLMs

Large Language Models sind eine Art der künstlichen Intelligenz, welche mithilfe probalistischer Methoden Text generiert. Im Grunde sind diese LLMs Text Vervollständigungsgeneratoren. Heutzutage basieren Chatbots auf sogenannten Large Language Models. LLMs sind in ihrem Kern nur 2 Dateien. Eine Datei die die sogenannten Gewichte des Neural Networks beinhaltet, welche beim Trainieren entstanden sind und eine Datei, welche den Code beinhaltet,damit man dieses Neural Network ausführen kann. Der Code lässt sich in ein paar hundert Zeilen Code schreiben, selbst ohne Abhängigkeiten. Der größere und aufwändigere Part sind die Gewichte. Die Gewichte entstehen, beim trainieren des Neural Networks, welches auf der Transformer - Decoder Architektur basiert, welche 2017 im Paper Älll you need is attention"vorgestellt wurde. Der genaue Stack den bspw. OpenAI heutzutage benutzt ist nicht bekannt, aber wir wissen, dass dieser ebenfalls auf der Transformer Architektur basiert. Wir sehen anhand der **??**, welche über Google Trends das Interesse am Wort "LLMïm Verlauf der letzten 5 Jahre darstellt, dass das Interesse an Large Language Models besonder stark im Dezember 2022 angestiegen ist und seitdem immer weiter steigt. Die Daten von Google Trends sind normalisiert zwischen 0 und 100 also können wird dadurch zwar keine genauen Angaben machen, wir sehen aber, dass das Wachstum stetig am steigen ist.

Im Grunde sind die LLMs die hinter Chatbots wie ChatGPT von OpenAI und LeChat von MistralAI sogenannte GPTs. Wenn man den Begriff aufteilt sieht man was dahinter steckt. GPT steht für Generative pretrained Transformer. Generative steht für die Art und Weise auf der diese LLM ihre Ausgaben generiert, Pretrained steht dafür, dass die LLM vor

ihrer Nutzung auf einer Datenmenge trainiert wurde und Transformer steht hier für die Transformer Architektur.

2.4.1 Transformer

Die Transformer Architektur wurde 2017 von Vaswani et al im Paper Ättention is all you need"vorgestellt. Vor dieser Architektur waren die besten Modelle Encoder-Decoder Architekturen (siehe 2.3.6), die mit dem Attention Mechanismus (siehe 2.3.7) ergänzt wurden und die auf Rekurrenten oder Convolutional Neuronalen Netzen aufgebaut haben.

Der Originale Transformer, der von Vaswani et al vorgestellt wurde, bestand ebenfalls aus einem Encoder und einem Decoder. Jedoch wurden keine Recurrent oder Convolutional Neuronalen Netze mehr genutzt. Die großen Neuheiten in dieser Architektur war das Konzept der Self-Attention, Parallelisierbarkeit, "Multi-Head Attentionünd das weg lassen der recurrent oder Convolutional neuronalen Netzen. **[?]**

Self-Attention

Während beim ursprünglichen Attention Mechanismus (siehe 2.3.7) Beziehungen zwischen dem Encoder und dem Decoder modelliert, ermöglicht Self-Attention es dem Modell, Beziehungen innerhalb einer einzelnen Sequenz zu erfassen.

Dies wird erreicht indem jedes Element einer Sequenz mit allen andere Elementen in Beziehung gesetzt wird. Dies hilft dabei den Kontext zu bestimmen.

Das heißt, durch die Self-Attention erkennt der Transfomer an welchen Position der Eingabe Sequenz er nachschauen muss um den aktuellen Vektor besser einbetten zu können.

Bei Self-Attention wird jeder Eingabe-Vektor durch drei Vektoren repräsentiert. Einem Query Vektor $\mathbf{Q}$, einem Key Vektor $\mathbf{K}$ und einem Value Vektor $\mathbf{V}$. Diese Vektoren werden erstellt indem der eingebette Eingabe Vektor $\mathbf{X}$ mit jeweils einer der Gewichts Matrizen W^Q, W^K, W^V multipliziert wird, welche während dem Training erstellt wurden.

$$Q = X\mathbf{W}^Q, K = X\mathbf{W}^K, V = X\mathbf{W}^V$$

Im nächstren Schritt werden die Skalarprodukte, die hier als Score bezeichnet werden, des Query Vektors des aktuell eingebetteten Vektors mit jeweils jedem Key Vektor der Token der aktuellen Eingabe berechnet.

$$Score_1 = Q_1 \cdot K_1$$

$$Score_2 = Q_1 \cdot K_2$$

$$...$$

$$Score_n = Q_1 \cdot K_n$$

Anschließend wird dieses Skalarprodukt durch die Wurzel der Dimensionen des Key Vektors d_k geteilt und auf das Ergebnis wird eine Softmax Funktion angewendet, die diese Werte normalisiert und dafür sorgt, dass sie summiert 1 ergeben.

$$ScaledScore = softmax(\frac{Score_1}{\sqrt{d_k}})$$

Als nächstes werden die Softmax Ergebnisse mit den Value Vektoren multipliziert und diese gewichteten Werte werden aufsummiert.

$$Attention = \sum_{i=1}^{n} ScaledScore_i V_i$$

Dies wird durch Matrix Berechnungen in einem Schritt erledigt mit der Formel:

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$$

Dies gibt uns den Attention Vektor für jeden Token.[**?**, **?**]

Multi-Head Attention

Multi-Head Attention wurde ebenfalls mit dem Self-Attention Mechanismus in die Transformer Architektur eingeführt. Multi-Head Attention wurde eingeführt, damit das Model

Dieser Mechanismus funktioniert so, dass man statt nur einem Attention Head in der Attention Layer mehrere Attention Heads in dieser sind. Im originalen Transformer wird mit 8 Attention Heads gearbeitet. jeder dieser Heads führt den Self-Attention Mechanismus aus, aber jeder hat unterschiedliche Gewichts Matrizen

$$W^Q, W^K, W^V$$

Jeder der Heads führt seine Berechnung mit unterschiedlichen Matrizen durch, was dazu führt, dass verschiedene Heads verschiedene Werte auf unterschiedliche Teile des Satzes legen können. Durch diesen Mechanismus kann es zu einem besseren Verständnis für Nuancen kommen.

Die finalen Attention Matrix z, die dem jeweiligen Token zugeschrieben wird, wird über eine weitere Matrix berechnet. Hierfür werden die Ergebnisse der Attention Heads $head_1, head_2, ..., head_n$ aneinander konkateniert und im Anschluss mit der Gewichts Matrix W_0 multipliziert.

$$z = Concat(head_1, head_2, ..., head_n)W_0$$

2.4.2 Tokenizer

Tokenizer sind ein wichtiger Part der Machine Learning Toolchain im Zusammenhang mit dem Lernen von menschlicher Sprache. Wenn man im Bereich des Natural Language Processing ein Machine Learning trainieren möchte stellt sich die Frage, wie man Texte aufteilen kann, damit das Modell, welches trainiert wird diese verarbeiten kann.

Die Tokenizer teilen die Texte so auf, dass das zu trainierende Modell komplexe Sätze besser aufteilen und verstehen kann. Der Tokenizer macht das, indem er den Text in sogenannte Tokens aufteilt. Ein Token kann ein ganzes Wort, ein Teil eines Worts oder ein Satzzeichen sein.

Dies ist ein wichtiger Teil des Natural Language Processing (NLP), da die Aufteilung der Trainingsdaten einen großen Einfluss darauf hat, wie gut die Large Language Model (LLM) funktioniert. Genau wie bei Compilern, die Programme in Tokens aufteilen, benutzen wir bei LLM Tokenizer, damit wir die Texte aufteilen können und die dabei entstandenen Tokens über verschiedene grammatikalische Regeln zusammensetzen können.

Eines der größten Probleme hierbei ist, dass verschiedene Sprachen verschiedene grammatikalische Regeln haben und somit verschiedene Arten des Zerlegens oder Zusammensetzens

brauchen. Ein weiteres Problem ist, dass das selbe Wort teils verschiedene Bedeutungen haben kann. Hierbei ist der Kontext wichtig und die Anzahl der Tokens, die das Modell erkennen kann. Man könnte hierbei alle möglichen Token aufnehmen, was aber dafür sorgen kann, dass das System overfitted beim Training. Dazu kommt, dass das die notwendigen Kosten zum Trainieren und für die Inference des Systems erhöhen würde [**?**, **?**].

2.4.3 Pretraining

Das Pretraining einer LLM setzt die Grundlage für die LLM. Für das Pretraining einer LLM werden sehr viele Daten benötigt und diese sollten eine hohe Qualität haben. Bessere Qualität dieser Daten, hilft der LLM später bessere Ausgaben zu generieren. Daher ist es wichtig eine gute Datenpipeline für die Trainingsdaten vorzubereiten, welche die Datenqualität sicherstellen.

Um ein etwas besseres Bild davon zu haben, wie viele Daten benötigt werden, wird Llama3 von Meta betrachtet. Llama3 wurde mit 15 Billionen Tokens trainiert.

Um einen Vergleich zu haben, wird der fineweb Datensatz, welcher auf Huggingface zur Verfügung steht, betrachtet. Dieser Datensatz hat ebenfalls 15T Token und hat eine Speichergröße von 47.6TB.

Für das Pretraining werden Daten aus den Bereichen benötigt, in denen die LLM später Dinge generieren soll. Ebenfalls die Sprache der Daten ist wichtig. Das Verhältnis der Menge der Daten zueinander beeinflusst ebenfalls die Performance der LLM.

Das Training ist der rechnerisch aufwändigste Teil der LLM Erstellung. Für das Training des Llama3 8B Parameter Models hat Meta 1.3 Millionen GPU Stunden gebraucht. Mit dem GPU Cluster, den Meta zur Vefügung hat, mit 24,576 H100 CPUs, hat es ca 2.2 Tage gedauert dies zu trainieren.[**?**, **?**, **?**]

In diesem Kontext gibt es auch verschiedene Unternehmen die Partnerschaften mit KI Unternehmen schließen und diesen Zugriff auf ihre Daten geben, damit sie besser in diesem Bereich werden. Ein Beispiel für so einen Zusammenschluss ist der Zusammenschluss von OpenAI und Stackoverflow, wobei OpenAI hofft, dadurch ein Model mit besseren Coding Kompetenzen zu erhalten und Stackoverflow hofft, dass ihre Nutzer dadurch mehr mit Inhalt auf der Seite interagieren [**?**].

2.4.4 Finetuning

Beim Finetuning einer LLM wird auf dem vortrainierten Model aufgebaut und das Model wird mit einem kleineren spezifischeren Datensatz weiter trainiert. Dieser Schritt ist rechnerisch weniger aufwändig als das vortrainieren und ist daher für mehr Unternehmen und Instiutionen zugänglich. Hinzu kommt, dass dieser Schritt weniger zeitaufwändig ist.

Um einen Vergleich für den Aufwand zu haben wird "DiscoResearch/Llama3-German-8Bäuf Huggingface betrachtet. Das Model, welches 1.3 Millionen GPU Stunden für das Pretraining gekostet hat, wird jetzt gefinetuned. DiscoResearch hat hierfür Zugriff auf den Supercomputer 42 von hessian.Ai bekommen. Das Model wurde von 128 GPUs innerhalb von ca 60 Stunden über 65 Milliarden Tokens gefinetuned. Dies heißt, es hat ca 7680 GPU Stunden gekostet. Dieser Vergleich zeigt, dass dieser Schritt weniger aufwändig ist als das Pretraining.

Teilweise wird Finetuning verwendet um sogenannte Instruct Modelle zu erstellen. Diese Instruct Modelle sind Versionen des Standard Models, die darauf finetuned wurden, Instruktion zu befolgen. Bei einem Model, wie Llama3 von Meta, ist die nicht finetuned Variante des

Models bereits in der Lage Instruktionen zu befolgen, wird aber durch ein kurzes Finetuning messbar besser in diesem Bereich [**?**].

Eine weitere Richtung in die Finetuning gehen kann ist das Trainieren eines Models auf einer neuen Sprache. Hierfür betrachten wir "DiscoResearch/Llama3-German-8Bäuf Hugginface. Eine auf deutsch fine-tuned Version von Llama3. Sie wurde mit Benchmarks veröffentlicht, die uns zeigen, dass die deutsche Leistung des Models sich durch das finetuning mittels 65 Milliarden Tokens verbessert hat. Die Englische Leistung ist etwas verschlechtert, aber vorallem beim hellaswag_de [**?**] Benchmark, sieht man, dass die Leistung sich messbar bessert. [**?**]

2.4.5 Inference & Generation

Sobald ein Model trainiert wurde und bereit ist benutzt zu werden, muss man es ausführen. Dieser Prozess nennt sich Inference. Um Inference ausführen zu können gibt es heutzutage verschiedene Möglichkeiten. Es gibt die Möglichkeit Inference über online APIs wie beispielsweise die Huggingface Inference API [**?**] oder ähnliche Services auszuführen, oder man kann die Modelle lokal ausführen.

Wenn man eine der APIs nutzt um ein Model zu nutzen, dann kann man Anfragen an die API schicken und das Ausführen des Models wird von den Betreibern der API übernommen.

Die andere Option, das Model lokal auszuführen, kann über verschiedene Methoden passieren. Eine Option ist es einen Service wie Ollama (siehe 2.9) [**?**] oder LM Studio nutzen [**?**], worüber man die Modelle downloaden und ausführen kann. Mit Ollama kann man das Model über die CLI ausführen und mit ihr kommunizieren und über LM Studio kann man das mittels der GUI machen. Beide Projekte basieren auf llama.cpp (2.8).

Eine andere Option, das Model lokal auszuführen, ist es die Gewichte der Modelle zu downloaden und diese mittels Code auszuführen. Hierfür kann man die Modelle teils direkt bei den Unternehmen downloaden wie bei Llama von Meta, aber eine andere Website auf der viele Modelle veröffentlicht werden, auf der man auch Vergleiche zwischen mancher dieser Modelle sehen kann ist hugginface.co (siehe 2.12).

Andrej Karpathy [**?**] hat gezeigt, dass man mit einen 15 Millionen Parameter Model von Llama2 in C ohne externe Libraries innerhalb von unter 1000 Zeilen Inferenz betreiben kann [**?**]. Modelle die heute veröffentlicht werden haben aber weit über 15 Millionen Parameter und daher ist die Inferenz mit so einem simplen Programm nicht sehr effektiv. Das Llama2 Model mit 7 Milliarden Parametern generiert auf einem M1 Macbook Air einen Token pro 30 Sekunden. Die Modelle in einer Milliarden Größe, brauchen aufgrund ihrer Größe ein paar Optimierungen.

Eine Möglichkeit Modelle schneller zu machen ist es sie zu quantisieren, dieser Prozess wird in 2.8 genauer erklärt.

Statt den Code für die Inferenz selbst zu schreiben gibt es auch noch die Möglichkeit verschiedene Framworks zu verwenden. Drei dieser Frameworks werden in **??** vorgestellt und erklärt.

Die Generation der LLMs kommt auf ihre Trainingsdaten an. Genau wie beim Training arbeiten die LLMs beim generieren ihrer Ausgaben in Tokens.

2.4.6 Limitationen & Halluzinationen

Reversal Curse [**?**, **?**]

2.4.7 Prompting

Ein Prompt im Bereich der LLMs ist die Texteingabe des Nutzers, welche die Ausgabe des LLMs bestimmt. Die Art und Weise der Eingabe kann von LLM zu LLM unterschiedlich sein. Bei Modellen, welche Bilder generieren, wie Midjourney oder DALLE-3 sind die Prompts eher beschreibend und bei Prompts für Chatbots, wie ChatGPT sind die Prompts eher Fragestellungen oder Anfragen [**?**].

Die Prompts können starken Einfluss auf die Qualität der Ausgaben des Systems haben. Es gibt verschiedene Methoden, die mit der Zeit gefunden wurden um die Ausgaben des Systems zu verbessern. Aufgrund dessen hat sich im Bereich der LLMs eine neue Disziplin namens Prompt Engineering gebildet. Innerhalb dieser geht es darum, Prompts so zu optimieren, dass das LLM System genau das macht was der Nutzer will.

Egal wie gut eine Prompt ist, manchmal kann eine LLM Applikation die gewollte Ausgabe nicht ausgeben. Dies kann verschiedene Gründe haben, wie zum Beispiel fehlende Informationen, die Informationen können fehlen, weil sie nicht mit in den Trainingsdatensatz der LLM aufgenommen wurden, obwohl es sie schon gab, aber es könnte auch sein, dass die Informationen erst veröffentlicht wurden, nachdem die LLM zu Ende trainiert wurde. Aufgrund dessen ist es wichtig, die Limitationen der LLM, die verwendet wird, zu kennen.

Es gibt zwar keinen eindeutigen Weg gute Prompts zu generieren, aber es gibt ein paar Best Practices, welche generell anerkannt sind. Im Prompt nur die Aufgaben mitteilen nennt sich Zero-Shot Prompting. Das heißt es werden keine Beispiele mitgegeben und manche LLMs sind in der Lage Zero-Shot prompts in manchen Aufgabenbereichen zu lösen. Der LLM im Prompt ein Beispiel für die gewollte Ausgabe mitzugeben nennt sich One-Shot Prompting und mehrere mitegeben heißt few-shot prompting. Die Qualität der gewollten Ausgaben lässt sich teils mithilfe von One-Shot oder Few-Shot Prompts statt Zero-Shot Prompts verbessern. Jedoch hat der Input Kontext eine begrenzte Größe und umso größer der Input ist, umso größer ist der computational(was heißt das auf deutsch?) Aufwand, daher ist das eine begrenzte Lösung, die nicht alle Probleme beseitigen kann. Ein paar der davon sind:

- Anweisungen an den Anfang oder das Ende des Prompts stellen.
- Spezifische Anweisungen und genaue Beschreibungen geben. Auch spezifisch und genau bei den gewollten Ausgaben sein.
- Anweisungen die der LLM mitteilen was sie tun soll, statt den Dingen die sie nicht tun soll, sind zu bevorzugen.

Es gibt auch ein paar fortgeschrittene Methoden die verwendet werden können. Hierzu gehören:

- Der LLM "kraftvoll"mitteilen, was man von ihr möchte.
- Gedankengang Prompting, bei der man die LLM, die einzelnen Schritte die sie gemacht hat und die Gründe dafür erklären lässt.[**?**]

Es gibt eine spezielle Art der Prompts namens System Prompt. Dieses System Prompt dient dazu das Verhalten der LLM in einem Rahmen zu steuern. Durch dieses System Prompt kann der LLM mitgeteilt werden, welche Rolle sie einnimt und unter anderem auch ihren Stil beeinflussen.

Backpropagation

2.5 Vektor Datenbanken

Vektor Datenkbanken sind Datenbanken, die hochdimensionale Vektoren speichern können. Diese Vektoren sind mathematische Repräsentationen von Attributen oder Merkmalen. Je nachdem wie granular die Daten, die abgespeichert werden sollen sind, kann es sein, dass die Daten tausende Dimensionen haben. Die Rohdaten werden mittels Einbettungsfunktion oder Transformationen in ihre Vektordarstellung umgewandelt und dann abgespeichert.

Diese Art der Datenbanken hat für den AI und Machine Learning Bereich ein paar Vorteile im Vergleich zu traditionellen Datenbanken.

Ähnlichkeitssuche: Traditionell Datenbanken brauchen klar definierte Kriterien für die Suche. Hier ist es nicht möglich eine Ähnlichkeitssuche, wie bei Vektordatenbanken durchzuführen. Bei Vektordatenbanken kann man mittels Ähnlichkeitssuche-Verfahren schnell ähnliche Datensätze finden und dann ausgeben.

Support für unstrukturierte und komplexe Daten: Traditionelle Datenbanken brauchen strukturierte Daten. Die Daten die man für Machine Learning benutzt sind oft unstrukturierte Daten, die verschieden Formate haben oder komplett verschiedene Datentypen sind, dies schließt auch Audio und Videodateien mit ein. Vektordatenbanken können diese Dateien durch die Vektordarstellung abspeichern und sind damit besser für diesen Fall geeignet.

Skalierbarkeit und Performance: Dadurch, dass Vektordatenbanken oft mit Big Data verwendet werden, gibt es einige Ansätze zum Verbessern der Performance und Skalierbarkeit. Ein paar der Methoden die verwendet werden sind Partitioning, Sharding und Caching. Wie genau diese Methoden funktioniert sind irrelevant für diese Arbeit und werden daher nicht weiter angesprochen.

2.6 Embedding Modelle

Embedding Modelle dienen dafür Daten in eine Vektorrepräsentation umzuwandeln. Diese Modelle sind in der Lage verschiedene Datenformate umzuwandeln. Unter anderem Text, Video und auch Bilder.

Diese Vektoren sind hochdiemnsional und sollen dadurch auch Nuancen und die Bedeutung der Wörter mit in die Vektorrepräsentation aufnehmen. Dadurch, dass der Text in die jeweilige Vektorrepräsentation umgewandelt wird, können nach der Umwandlung verschiedene mathematische Verfahren auf diesen Vektor angewendet werden. Dazu zählen Verfahren wie Semantische Suche und Semantische Nähe. Dies sorgt ebenfalls dafür, dass das Arbeiten mit den Daten schneller ist.

Embedding Modelle werden in LLM Applikationen dafür benutzt, Daten in ihre Vektorrepräsentation umzuwandeln und diese Vektoren zusammen mit den dazugehörigen Daten in einer Vektordatenbank zu speichern. [**?**, **?**, **?**]

Semantische Suche

Bei der semantischen Suche handelt es sich um ein Verfahren, welches dafür genutzt wird Daten semantisch zu vergleichen. Die Daten werden hierfür in ihre Vektordarstellung umgewandelt und dann werden die Vektoren miteinander verglichen und die top k Vektoren mit der höchsten Semantischen Nähe werden zurückgegeben.

Hierfür gibt es verschiedene Verfahren, drei Verfahren, die oft verwendet werden sind:

- Euklidischer Abstand

$$d(\mathbf{p}, \mathbf{q}) = \sqrt{\sum_{i=1}^{n} (p_i - q_i)^2} \tag{2.1}$$

- Kosinus-Ähnlichkeit

$$\cos(\theta) = \frac{\sum_{i=1}^{n} p_i q_i}{\sqrt{\sum_{i=1}^{n} p_i^2} \sqrt{\sum_{i=1}^{n} q_i^2}} \tag{2.2}$$

- Manhattan-Metrik

$$d(\mathbf{p}, \mathbf{q}) = \sum_{i=1}^{n} |p_i - q_i| \tag{2.3}$$

Mittels einer dieser oder eines anderen Verfahrens wird dann die Semantische Nähe berechnet und es werden die passenden Dateien, falls es diese gibt, zurückgegeben. Dies hilft es den LLM Applikationen dabei, passende Daten schnell abzurufen [**?**, **?**, **?**].

2.7 RAG

Eines der größten Probleme der LLMs heutzutage ist, dass die Informationen, die sie zum beantworten für Fragen benutzen, teilweise veraltet sind oder sehr allgemein. Ein weiteres großes Problem ist die im Abschnitt 2.4 genannte Halluzination, die passieren kann, wenn eine LLM eine Frage zu der es eigentlich nichts weiß, beantwortet. Eine Technologie, die versucht, dieses Problem zu beseitigen, ist die sogenannte Retrieval Augmented Generation. Die Retrieval Augmented Generation soll das parametrische Gedächtnis der LLM mittels eines nicht parametrischen Gedächtnisses erweitern.[**?**]

Retrieval Augmented Generation löst ein paar Probleme vor denen LLMs heutzutage stehen, ist aber keine perfekte Lösung und muss sich auch seinen eigenen Problemen stellen. In diesem Zusammenhang gibt kann man heute RAG in verschiedene Stufen einordnen.[**?**]

2.7.1 Naive RAG

Naive RAG ist die traditionelle Herangehensweise and das Problem der RAG. Sie besteht aus drei Stufen. Indexing(Indizierung??), Abfrage und Generierung.

Beim Indizieren werden die rohen Eingabedaten gereinigt und extrahiert. Dies ist wichtig, da es verschieden Formate haben kann, bei Textdaten gibt es bspw. PDF Dateien, Markdown oder HTML. Diese Texte werden nach der Reinigung in Plain Text umgewandelt und dieser Plain Text wird dann in kleinere Teile unterteilt . Diese kleineren Parts werden dann über ein Embedding modell in eine Vektordarstellung umgewandelt und dann in einer Vektordatenbank abgespeichert.

Bei der Abfrage erhält das RAG System eine Anfrage und benutzt das selbe Embedding Modell, wie in der Indizierungsphase, um die Abfrage in eine Vektordarstellung umzuwandeln. Im Anschluss wird durch einen Semantischen Vergleich nach den Top-k Ergebnissen gesucht und diese werden dann ausgegeben.

Bei der Generation bekommt die LLM dann die Abfrage und die ausgegeben Ergebnisse in einem Prompt zurück und generiert daraus dann eine Antwort. Was genau hier beachtet wird kommt auf die LLM, das Kontext Fenster und den benutzten Prompt an[**?**].

Wie in 2.7 bereits erwähnt gibt es hierbei ein paar Probleme. Das Hauptproblem, welches hier besteht ist die der Abfrage. Bei der Abfrage gibt es oft Probleme, welche mit der

Genauigkeit zu tun haben. Dementsprechend kann es passieren, dass das RAG System falsche oder unpassende Informationen zurück gibt. Aufgrund der Probleme bei der Abfrage entstehen dann ebenfalls Probleme bei der Generation der Antwort. Was dazu führt, dass die Qualität der Antworten sinkt. Dadurch sinkt dann ebenfalls die Verlässlichkeit auf das System[**?**].

2.7.2 Advanced RAG

Advanced RAG wurde entwickelt um den Problemen der naiven RAG entgegenzuwirken bzw. sie zu beseitigen. Dementsprechend werden hier einige Dinge geändert. Vorallem in der Stufe der Abfrage wurde hier einiges geändert. Um die Qualität zu erhöhen wurden der Abfrage pre- und post Strategien hinzugefügt. Dazu wurde der Indexing Prozess angepasst. Beim Indexing wird nun Metadata mit eingebunden und der Text wird genauer unterteilt. Pre-retrieval: Beim pre-retrieval wird vorallem darauf geachtet die Anfrage so klar wie möglich zu machen. Hierfür werden Metadaten angehängt, Anfragen umgewandelt, Anfragen erweitert und noch einige weitere Dinge abgeändert. Das Ziel hier ist es die Anfrage zu ändern, sodass sie so klar wie möglich ist und sie für Retrival zu optimieren.

Post Retrieval: Nachdem der Kontext nach dem Retrieval Prozess abgefragt wurde, wird versucht die abgerufenen Daten möglichst optimal mit der Anfrage zu verbinden. Hierfür werden vorallem das komprimieren und das neu Anordnen des Kontexts verwendet. Die wichtigsten Dokumente werden an eines der Enden der Anfrage geschoben und es wird versucht die wichtigsten Informationen heraus zu suchen und nur diese zurück zu geben. Dadurch sollte die LLM genug Informationen erlangen, aber nicht zu viele und die Antworten sollten besser werden[**?**, **?**].

2.8 llama.cpp

Ein Problem, welches im Abschnitt 2.4.5 bereits angesprochen wurde, ist, dass es schwierig sein kann, für Personen oder Unternehmen, die nicht über die nötige Hardware verfügen, LLM Inferenz zu betreiben. Desto größer die LLM, die benutzt werden soll, wird, desto aufwändiger werden die Anforderungen an die Hardware. Ein großes Open Source Projekt, welches sich mit diesem Problem auseinander setzt ist llama.cpp, gestartet von Georgi Gerganov auf Github.

Dieses Projekt hat zu Beginn viele Nutzer angezogen, da man hier, ohne spezielle Hardware, Inferenz betreiben konnte. Die anderen Bibliotheken, die hierfür zu Verfügung standen, wurden oft in Verbindung mit Closed-Source Bibliotheken, wie zum Beispiel Nvidias Cuda entwickelt und waren dadurch nur über bestimmte Hardware zu benutzen.

Zu Beginn wurde das Projekt hauptsächlich auf Inferenz auf der CPU ausgelegt. Dies gab den Nutzern zwar die Möglichkeit auf ihren Maschinen ohne GPU Inferenz zu betreiben, aber durch fehlende GPU Einbindung hatte dies Begrenzungen in der Geschwindigkeit und der Größe der Modelle, die benutzt werden konnten.

Mit der Zeit wurde dem Projekt auch die Möglichkeit hinzugefügt, GPUs für die Inferenz zu benutzen, wodurch die Nutzer in der Lage sind schnellere Inferenz mit größeren Modellen auszuführen.

Damit die verschiedenen Modelle über llama.cpp nutzbar sind, gab es hier zu Beginn das GGML format. Dies wurde ebenfalls von Gerganov im Zusammenhang mit seinen Projekten entwickelt.Ein Vorteil hierbei war, dass das GGML Format das speichern von quantisierten

Modellen erlaubt. Quantisierte Modelle haben hierbei den Vorteil, dass die Präzision der einzelnen Parameter von bspw. 32 Bit Gleitkommazahlen auf eine niedrigere Genauigkeit, wie bspw. 8 Bit Gleitkommazahlen konvertiert wurde. Der Vorteil hier war, dass die Inferenz der einzelnen Modelle hier weniger intensive Rechenarbeiten erfordert haben und auch durch die kleinere Größe auch weniger Speicher benötigt. Dadurch war es einfacher möglich, die Inferenz auf der CPU durchzuführen.

Jedoch hatte diese Format einige Probleme. Zum einen hat jede Architektur ihr eigenes Skript benötigt, damit sie in das GGML Format konvertiert werden kann, dazu kam, dass in der Datei nicht erkennbar war, für welche Architektur das Modell war. Heutzutage benutzt llama.cpp ein neues Format, namens GGUF, welche einige der Probleme mit GGML beseitigen soll. Ein großer Vorteil von diesem Format ist, dass das Modell nun alle Informationen die zum laden des Modells notwendig sind beinhaltet und es müssen keine zusätzlichen Informationen mehr vom Nutzer gegeben werden. Unter anderem beinhaltet die Datei nun die Architektur des Modells

[**?**, **?**].

2.9 Ollama

Ollama ist ein Open Source Projekt, welches seinen Nutzern dabei helfen soll, auf der Plattform LLMs schneller auf ihren lokalen Maschinen zu starten und mit diesen zu interagieren. Das Backend das Ollama unterstützt ist llama.cpp und es soll das Downloaden, Installieren und das Interagieren mit den LLMs einfacher machen.

Ollama hat seine eigene Modellbibliothek über die Nutzer LLM Modelle downloaden können und diese dann direkt nutzen. Wenn ein Modell nicht in der Bibliothek zu finden ist, kann man Modelle mittels Modelfile selbst importieren. Dies kann man mit Modellen im GGUF Format, welches in 2.8 genauer erklärt wurde, gemacht werden und mit sogenannten safetensor Dateien[**?**]. Das Safetensor Format wurde von Huggingface eingeführt um die Unsicherheiten des vorher genutzten Pickle Formats zu beseitigen. Eines der größten Probleme dieses Formats war es, dass beim deserialisieren eines Pickle Objekts boshafter Code ausgeführt werden konnte, da Pickle den unterschied zwischen boshaften und nicht boshaften Code nicht erkennen kann.[**?**] Mit dem neuen Safetensor Format konnte vorallem dieses Problem beseitigt werden.

Ollama kann Safetensor Dateien für die drei Architekturen LlamaForCausalLM, MistralForCausalLM und GemmaForCausalLM ausführen. Wenn man ein Modell hat, welches nicht das GGUF Format oder das Safetensor Format für eine der drei genannten Architekturen ist dann muss man dieses Modell in GGUF umwandeln. Dafür kann man in das llama.cpp Github Projekt und das convert script benutzen.

Im Anschluss kann man dann das Modell mithilfe einer Modelfile importieren. Innerhalb dieser Modelfile kann man ebenfalls noch weitere Dinge vorher festlegen, wie zum Beispiel das System Prompt für das Model[**?**].

2.10 Ragas

2.11 AI Frameworks

Dadurch, dass künstliche Intelligenz nun ein größeres Thema in den Augen der Öffentlichkeit ist und es durch Bibliotheken wie llama.cpp einfacher geworden ist, für Personen ohne besondere Hardware LLMs zu nutzen, gibt es immer mehr Entwickler, die Applikationen, die LLMs nutzen, entwickeln wollen.

Da es immer mehr Applikationen dafür gibt, gibt es auch einige KI Frameworks, die das Arbeiten mit diesen Modellen vereinfachen sollen. Ein großer Fokus bei den meisten der KI Frameworks liegt darauf den Nutzern das Arbeiten mit eigenen Daten einfacher zu machen indem beispielweise eine RAG Pipeline genutzt wird. Hier werden einige der größten Frameworks vorgestellt.

2.11.1 Haystack

Haystack ist ein Open Source Framework, welches LLM Applikationen erstellen kann. Der Fokus hier liegt vorallem auf RAG Applikationen und Such Systemen, welche mit großen Mengen an Daten arbeiten können.

Bei Open Source Frameworks ist die Community einer der wichtigsten Faktoren. Umso mehr Teilnehmer an einem Open Source Projekt es gibt, umso mehr Unterstützung gibt es und umso mehr Fehler fallen den Leuten auf, die dann beseitigt werden können. Haystack hat 14.3k Sterne auf Github, zum Zeitpunkt dieser Arbeit. Das macht es zum kleinsten Projekt auf dieser Liste.

Haystack kann direkt Modelle von Huggingface, OpenAI und anderen Quellen beziehen und diese ausführen. Dazu kann man Modelle nutzen die auf Hosting Plattformen, wie Azure oder Sagemaker gehostet werden.

Die zwei Konzepte die hierbei am wichtigsten sind sind Komponenten und Pipelines. Komponenten sind Blöcke, welche Aufgaben durchführen können. Aufgaben, die eine Komponente ausführen kann, sind zum Beispiel Text Generation und das Erzeugen von Einbettungen.

Eine Pipeline besteht aus verschiedenen Komponenten. Man setzt diese Komponenten zusammen und erhält dadurch eine Pipeline. Dadurch kann man den Datenfluss innerhalb seiner Applikation genau beeinflussen.

Die Programmiersprache mit der Haystack arbeiten kann ist Python[**?**, **?**].

2.11.2 Langchain

Langchain ist ein weiteres Open Source Framework, welches für das Erstellen von LLM Applikationen erstellt wurde. Im Gegensatz zu Haystack ist Langchain nicht nur auf das Erstellen von RAG Applikationen und Such Systemen fokussiert. Langhchain versucht möglichst vielseitig einsetzbar zu sein. Öne Framework. Infinite Use Cases"werben sie auf ihrer Homepage.

Langchain ist, in Hinsicht auf die Github Sterne das größte Framwork, welches hier vorgestellt wird. Zur Zeitpunkt des Schreibens dieser Arbeit, hat das Projekt 87.6k Sterne auf Github.

Für die Entwicklung liegt der Fokus bei Langchain auf Chains. Welche ähnlich wie Pipelines den Datenfluss bestimmen. Man erstellt hier die einzelnen Teile die man braucht,

wie einen Retriever der mit einer Vektor Datenbank verbunden ist und einer LLM. Diese verbindet man dann noch mit der Eingabe des Nutzers und hat dann eine fertige Chain, die man dann einsetzen kann, wie man möchte. Eine Möglichkeit wäre es diese per RestAPI verfügbar zu machen.

Dazu hat gibt es das Paket Langserve, welches darauf ausgelegt ist, einfach Chains als RestAPIs zu veröffentlichen. Dazu kommt ein Paket namens Langsmith, welches zum Monitoring und zum Evaluieren von LLM Applikation dient. Langsmith ist nicht Open Source und kostet für Teams oder für Hobby Entwickler ab einer gewissen Menge an Nutzungen ein monatliches Abbonement.

Langchain ist ein Framwork, welches für Javascript und für Python zur Verfügung steht. [**?**, **?**]

2.11.3 LlamaIndex

LlamaIndex ist ein weiteres Open Source Framework, das entwickelt wurde, um die Erstellung von LLM Applikationen zu erleichtern.

Zum Zeitpunkt des Schreibens dieser Arbeit hat das Projekt 32.8k Sterne auf GitHub und ist damit das zweitgrößte Projekt, welches hier vorgestellt wird, aber es ist trotzdem weniger als halb so groß, wie Langchain.

Bei LlamaIndex liegt der Fokus auf dem Arbeiten mit Daten. Deshalb ist einer der Fokus Punkte des Frameworks der Index. RAG Use Cases in LlamaIndex benutzen in ihrem Kern den Index als Kernkonzept. Indexe werden aus Dokumenten erstellt und sind im Grunde Knoten-Objekte, welche Teile von den Originaldokumenten respräsentieren.

LlamaIndex hat eine Plattform namens LlamaCloud, welche einen durch generative künstliche Intelligenz angetriebenen Parser hat. Dieses Projekt heißt LlamaParse und lässt sich mit LlamaIndex integrieren. LlamaParse kann man auch ohne die LlamaCloud nutzen. Wenn man die LlamaCloud nutzt hat man noch die Option seine RAG Daten dort über eine API zu speichern. Über die API kann man sich direkt mit verschiedenen Datenquellen verbinden.

LlamaIndex ist ein Framework, welches, wie Langchain, für Javascript und für Python zur Verfügung steht[**?**, **?**, **?**, **?**].

2.12 Huggingface / MTEB

Huggingface ist eine Plattform, auf der verschiedene Daten gehostet werden, Diskussionen stattfinden und KI Apps zu finden sind.

Auf dieser Plattform werden Daten in Git basierten Repositories gelagert. Zu den Daten die hier zu finden sind, gehören unter anderem Open Source LLM Modelle, Open Source Embedding Modelle und Datensätze zum trainieren von solchen Modellen. Es gibt die Möglichkeit KI-basierte Demo Apps in sogenannten Spaces auszuprobieren oder zu hosten. Hinzu kommt es die Möglichkeit Beiträge zu veröffentlichen, ähnlich wie auf Social Media Plattformen, wie X oder Threads.

Neben den Demo Apps die in Spaces zu finden sind, werden auch Ranglisten veröffentlicht. Eine der größten Ranglisten ist die MTEB Rangliste. MTEB steht für Massive Text Embedding Benchmark. Diese Rangliste wurde für Einbettungsmodelle erstellt und wird regelmäßig geupdated. Die Modelle werden hier in 8 verschiedenen Aufgabenbereichen verglichen.

Ein paar der Aufgabenbereiche sind Clustering, Semantische Nähe und Zusammenfassungen. Die Datensätze wurden so ausgewählt, dass es eine verschiedene Länge an Texten gibt, damit die verglichen werden kann, wie die Modelle mit kurzen und auch mit langen Texten umgehen können. Aktuell ist das beste Modell im Durchschnitt in dieser Rangliste das Modell "SFR-Embedding-2_R" von Salesforce Research [**?**, **?**].

2.13 Google

2.13.1 Google Cloud

Dadurch, dass die Techdivision ein Google Partner ist und intern versucht möglichst viele Produkte von Google zu nutzen

Chainlit

2.13.2 Google Chat

Google Chat ist eine von Google entwickelte Chatplattform. Diese Plattform hat 2017 Google Hangout zusammen mit Google Meet ersetzt. Zu Beginn war das Projekt nur im Zusammenhang mit Google Workspace erhältlich und somit war es eine Chatplattform die hauptsächlich für Teams und Unternehmen benutzbar war. 2021 wurde die Plattform im Early Access auch für normale Benutzer erreichbar gemacht und ist seit 2022 für jeden benutzbar.

Diese Plattform ist die Plattform, die die Techdivision hauptsächlich intern zur Kommunikation benutzt. Es gibt auf der einen Seite 1 zu 1 Chat in der eine Person mit nur einer anderer Person kommuniziert und auf der anderen Seite gibt es Innerhalb dieses Services die Möglichkeit sogenannte Spaces zu erstellen, welche Gruppenchats ähneln. Ein Teil dieser Gruppenchats sind die in 2.1 genannten Supportchats in die der Chatbot final eingebaut werden soll.

3 Anforderungen

Es gibt sechs Punkte die laut Experteninterviews gefordert sind.

1. Der Bot soll selbst gehostet werden Das größte Problem, welches beim Betreiben des Chatbots beachtet, ist die DSGVO. Datenschutz ist ein großes Thema bei Unternehmen und vor allem, bei Unternehmen die mit Kundendaten arbeiten müssen oder auch Betriebsgehemnisse, die nicht an die Öffentlichkeit weiter gegeben werden sollen. Die Techdivision ist ein IT-Dienstleister und somit ist dies auch ein Bereich der bei der Konzeption und Entwicklung dieses Chatbots beachtet werden muss. Aufgrund dieser Sorgen wird gefordert, dass der Chatbot von der Techdivision gehostet wird, was bedeutet es dürfen keine APIs von externen LLM-Systemen hierfür verwendet werden, die von anderen Unternehmen gehostet werden, wie beispielsweise die ChatGPT API.

2. Der Bot soll in einer Chatgruppe laufen, damit IT-Supportmitarbeiter eingreifen können, falls nötig Auch, wenn die LLMs heutzutage weit entwickelt sind, können diese die falschen Informationen weiter geben, oder komplett neue halluzinieren und diese dann dem Supportsuchenden weiter geben. Es könnte auch sein, dass der Bot dem Mitarbeiter überhaupt nicht helfen kann. Damit in diesen Fällen ein Supportmitarbeiter eingreifen kann, sollte der Bot in einer Chatgruppe laufen, von der ein Mitarbeiter des Support Teams ebenfalls Teil ist.

3. Der Bot soll Unternehmensdaten verwenden Da die Bots nicht von Haus aus Zugriff auf spezifische Informationen der Techdivion Zugriff haben, soll der Bot auf Unternehmensdaten zugreifen können. Dadurch kann der Bot spezifischer Hilfe leisten und nicht nur allgemeine Hinweise geben, die im Internet zu finden sind.

4. Der Bot soll mit der Zeit auf neues Wissen zugreifen können Da sich die Unternehmensdaten mit der Zeit ändern und auch neue Datenquellen hinzukommen können, sollte der Bot in der Lage sein mit der Zeit auf neues Wissen zuzugreifen.

5. Der Bot soll auf Deutsch arbeiten Der Großteil der Techdivion ist deutschsprachig. Die meisten Teile von Datenquellen, wie beispielsweise Confluence sind ebenfalls auf Deutsch. Es gibt ein paar englischsprachige Mitarbeiter, daher wäre ein Bilingualer Bot der beste Fall, da aber die meisten Deutsch sind, wäre Deutsch die bessere Option, falls es nur eine Sprache sein kann.

6. Der Bot soll Anfragedaten speichern können Dies ist eine optionale Anfrage des IT-Supports. Der Bott soll Anfragedaten speichern können, damit eingesehen werden kann, ob zu manchen Themen viele Fragen gestellt werden oder manche Personen besonders viele Fragen stellen. So könnte, falls notwendig zu einem bestimmten Thema ein Workshop angeboten werden oder es könnte ein Kurs mit einer Person geben, in der diese Person alles, was sie will erklärt kriegt.

4 Konzeption

Bevor mit der Entwicklung begonnen werden kann, muss das Konzept erstellt werden. Zur Konzeption gehört in dieser Arbeit, die Auswahl der technischen Werkzeuge, die Programmiersprache, die Umgebung und die Modelle die benutzt werden sollen.

4.1 Auswahl der Chatmodelle

Nach der Analyse beginnt die Konzeption. Die erste Frage, die beantwortet werden muss, ist, welche Modelle werden benutzt. Es werden Modelle an zwei Stellen benötigt. Die erste Frage ist, welches Modell soll generell für den Chatbot benutzt werden. Hier gibt es ein paar Möglichkeiten. Die Anforderung die hier relevant ist, ist die Anforderung Nummer 1. Da der Bot lokal laufen sollte, werden keine Modelle benutzt, die nur über eine externe API erreichbar sind, wie beispielsweise OpenAIs ChatGPT, um Datensicherheit sicherzustellen. Also ist die einzige Option ein Modell zu downloaden und mit diesem zu interagieren. Hier gibt es zur Zeit fast täglich neue Modelle. Viele verschieden Unternehmen und Instiutionen haben angefangen ihre eigenen Modelle oder abgewandelte Versionen größerer Modelle zu trainieren und zu veröffentlichen. Zu den bekanntesten Open Source Modellen gehören die Modelle:

1. Mistral von Mistral AI
2. Llama von Meta AI
3. Gemma von Google

In dieser Arbeit werden 5 verschiedene Modelle miteinander verglichen. Die meisten der Open Source Modelle basieren aber auf den Modellen von Meta AI und Mistral AI, das heißt sie sind finetuned Versionen von diesen. Ein Hub auf dem viele der Modelle veröffentlicht werden ist hugginface.co. Hier werden viele Open Source Modelle veröffentlicht und man kann sie downloaden und verschiedene Benchmarks, etc. einsehen.

Einige der größeren Modelle wie beispielsweise Gemma und Llama3 wurden mehrsprachig trainiert und verstehen in ihrer Grundversion bereits teilweise beide Sprachen, die wir brauchen. Aus Speicher, Leistungs und Generationtime Gründen werden in dieser Arbeit nur Modelle unter 12B Parametern miteinander verglichen. Obwohl die oben genannten Modelle Deutsch in einem gewissen Rahmen bereits verstehen, ist in der 8B und 7B Variante der Modelle nicht sichergestellt, ob sie alles verstehen, durch die kleinere Trainingsdaten- und Parametermenge. Dementsprechend werden, wo möglich, finetuned Versionen die auf Deutsch trainiert wurden verwendet.

Zwei Organisationen die sich damit beschäftigen, Open Source Modelle auch auf Deutsch verfügbar zu machen nennen sich DiscoResearch [**?**] und LAION LeoLM [**?**]. LAION LeoLM hat mithilfe des Supercomputers Die Modelle die innerhalb dieser Arbeit miteinander verglichen werden sind

- Llama3 von MetaAI in der Version Llama3-DiscoLeo-Instruct-8B-v0.1 von DiscoResearch
- Gemma von GoogleAI in der Version SauerkrautLM-Gemma-7b von VAGOsolutions
- Mistral2 von MistralAI in der Version em_german_leo_mistral von jphme
- Phi-3 Von Microsoft in der Version Phi-3-small-8k-instruct von microsoft

4.2 Auswahl der Embedding Modelle

Nach der Auswahl des Chatmodels geht es um die Auswahl der nächsten Modelle. Es wäre hier möglich das extra Einbettungsmodell weg zu lassen und es mit den Chatmodellen einbetten zu lassen. Damit der Vergleich zwischen den verschiedenen Chatmodellen nicht durch die Einbettungen beeinflusst wird, wird für alle Chatmodelle das selbe Einbettungsmodell verwendet.

Ebenfalls, wie bei den Chat Modellen werden hier Open Source Modelle benötigt, welche auf einem eigenen Server gehosted werden können. Die Gründe sind die selben, wie bei den in 4.1 erwähnten Chat Modellen.

Hierfür wird das Massive Text Embedding Leaderboard (MTEB) auf Huggingface betrachtet. Das MTEB Leaderboard sortiert Embedding Modelle nach ihrem durchschnittlichen Benchmark. Es gibt zur Zeit der Verfassung dieser Arbeit keine Möglichkeit nach deutschen Modellen zu filtern. Daher wird das aktuell generell beste mehrsprachige Modell verwendet. Aktuell ist das das "multilingual-e5-large-instruct"Model, von intfloat, wer Teil von Microsoft Research in Asien ist. Dieses Model baut auf dem Model *lm-roberta-large"von FacebookAI auf, bevor es MetaAI wurde. Das Model wurde finetuned indem es mit 1 Milliarde Datensätzen kontrastiv vor trainiert wurde und im zweiten Schritt mit den Datensätzen, die für das E5-Mistral Model in der Arbeit Ïmproving Text Embeddings with Large Language Models"verwendet wurden, trainiert.

4.3 Prompting

4.4 Auswahl der Vektor Datenbank

Für die Vektordatenbank wird eine Open Source Vektordatenbank bevorzugt. Dazu wird eine LangChain Integration gefordert.

Wir werden hierfür eine self hosted Instanz von Milvus

4.5 Server

4.6 Chainlit

4.7 Verbindung zu Google Chat

4.8 Architektur des Systems

Nachdem die Auswahl des Development Tools und des Models abgeschlossen ist, ist die nächste Frage, welche Architektur das gesamte System haben soll. Die Schnittstelle, über die der Nutzer später mit dem Chatbot kommunizieren soll, ist bestenfalls eine Schnittstelle zu Google Chat. Dies ist aber der unwichtigste Part an diesem Punkt. Notfalls kann man auch ein anderes Chat System anbinden, daher kümmern wir uns als erstes um die anderen Teile der Architektur.

4.8.1 Verbindungen

Abbildung zeigt den konzeptionellen Aufbau des Systems. Der Chatbot wird auf einem REST-Server laufen, welcher auf einem dedicated Server läuft, der genauer in **??** erklärt wird. Der Rest Server ist ein Python LangServe Server, der auf FastAPI basiert.

Die Python Applikation ist mit einer Vektordatenbank verbunden, die auf dem selben Server in einem Docker Container in Betrieb genommen wird. Die Daten die auf den Docker Container geladen werden kommen von der Confluence Instanz der Techdivision. Um die Daten auf die Datenbank zu laden gibt es ein weiteres Python System, welches über einen Cron Job, alle 24 Stunden die neuen Daten von der REST-API des Confluence Systems abfragt und diese in die Datenbank überträgt.

Die Anbindung der Vektordatenbank und der Aufbau des RAG-Systems wird in 4.8.3 erklärt.

Das System wird erst ohne Anbindung an das externe Chat-System gebaut und wird daher erst mit Chainlit gebaut. Sobald hierbei alles abgeschlossen ist, wird die Verbindung zum anderen Chat-System hergestellt. Die Python Applikation des Chatbots ist dann mit der Google Chat API verbunden, welche eingestellt ist, über HTTPS-Server Instruktionen zu erhalten. Wie genau dies eingestellt ist und welche anderen Optionen es gibt wird in **??** erklärt.

Die Nachrichten, die das System erhält werden ebenfalls in der Datenbank abgespeichert, wo sie nachher analysiert werden können, um die Anforderung A6 zu erfüllen. Damit wäre die grobe Architektur beschrieben.

4.8.2 Chatbot

Bei der Auswahl der Konzeption für den Chatbot ist die Frage, welches Framework verwendet werden soll. Die drei Stück die zu Auswahl stehen wurden in 2.11 vorgestellt. Haystack, Langchain und Llamaindex. Vom Grundprinzip her könnte man jedes der drei Frameworks verwenden um einen nützlichen Chatbot mit RAG zu bauen. Bei allen drei gibt es die Möglichkeit mit RAG zu arbeiten und es ist ein Punkt auf den sich alle drei Stück fokussieren.

Einer der wichtigsten Punkte in Hinsicht auf das Framwork, welches verwendet werden soll, ist die Langlebigkeit des Projekts und ebenfalls die Menge an Dokumentation und

Support. Da die Welt der LLM Systeme sich aktuell schnell ändert und viele neue Erkentnisse in der Forschung gesammelt werden, verändern sich auch die AI Frameworks mit den LLM Systemen zusammen [**?**]. Da das System möglichst langlebig sein soll und im besten Fall nur die einzelnen Bauteile, wie beispielsweise die LLM, mit der Zeit ausgetauscht werden sollten, ist es wichtig zu beachten, dass es ein Framework ist, welches möglichst lange bestehen bleibt und das Austauschen von einzelnen Teilen ohne großen Aufwand erlaubt. Dies hilft auch bei der Entwicklung bzw. Weiterentwicklung, da dadurch verschiedene Setups ausprobiert werden können ohne große Veränderungen durchführen zu müssen. Die Community von Langchain ist die größte von den drei Framworks die in dieser Arbeit betrachtet werden, daher ist anzunehmen, dass der Support den man durch die Community erhält und die Menge an Dokumentation, die man finden kann hierfür am größten sind. Eine Github Suche nach Haystack gibt uns 1.7k Ergebnisse. Eine Suche nach Langchain gibt uns 25k Ergebnisse. Und eine Suche nach LlamaIndex gibt uns 1.3k Ergebnisse. Eine Google Trends Analyse in diesem Bereich wäre vermutlich nicht sehr aussagekräftig, da der Begriff Haystack von verschiedenen Unternehmen benutzt wird und dazu noch ein normal gebrauchtes Wort ist, daher werden wir uns an den Github Ergebnissen orientieren, da auch, wenn dies möglicherweise nicht alle Projekte auflistet, es uns zumindest eine Orientierungsrichtung gibt. Langchain hat hier den absoluten Vorteil.

Das Austauschen von einzelnen Teilen des Systems scheint mit Langchain am einfachsten zu sein, aber ist je nachdem, was man austauchst mit allen drei Framworks relativ einfach. Wenn man eine größere Änderung durchführen will, wie beispielweise das Ändern von einem simplen Text-System zu einem multimodalen System, welches ebenfalls mit Bildern arbeiten kann dann würde es relevant aufwändige Änderungen in allen drei Frameworks erfordern. Daher nehmen wir hier an, dass es bei allen drei den gleichen Aufwand kosten würde.

Hinzu kommt, dass Langchain bereits ein Datenladensystem für Confluence hat, welches das System ist welches an die Vektordatenbank angebunden werden soll. Dadurch, dass Langchain vielseitiger einsetzbar ist, als LlamaIndex und und Haystack, kommt der Vorteil dazu, dass Erfahrung mit dem Framework gesammelt werden kann und im Anschluss weitere Projekte, die über Chatbots und RAG Systeme hinaus gehen entwickelt werden könnten. Aus diesen Gründen wird Langchain für diese Projekt ausgewählt.

4.8.3 Datenupdates

Hierbei geht es um das Thema des Hinzufügens von neuem Wissen zum Chatbot. Das Problem bei einem trainierten Modell ist es, dass es keine neue Information ohne ein neues Training in seine Wissensdatenbank aufnehmen kann. Da sich die Daten in den jeweiligen APIs ständig ändern können, stellt das ein Problem dar. Hier ist aber nicht nur neue Information ein Problem, sondern auch geänderte alte. Woher soll der Chatbot wissen, dass sich irgendeine Information, die bereits in der Wissensdatenbank vorhanden ist, geändert hat. Für das Problem von neuen Informationen gibt es hier zwei Optionen. Das Modell, auf dem der Chatbot aufbaut, könnte neu trainiert werden, mithilfe der neuen Informationen. Hier sind die Fragen, wie oft es neu trainiert werden soll, wie lange dieser Prozess dauert und wie viel dieser Prozess kostet. Wir können das Modell nicht bei jedem neuen Update neu trainieren. Das würde zu lange dauern, aber das heisst dass es einen Zeitraum gibt in dem der Bot nicht die aktuellsten Informationen hat. Wir benutzen hier die vorher bereits erwähnte RAG in Kombination mit einer Vektor Datenbank.

Welche RAG Variante benutzt wird, wird durch unser Framework, welches in 4.8.2

ausgewählt wurde, bestimmt. Jedoch gibt es verschiedene Vektordatenbanken für die sich entschieden werden muss.

4.8.4 Server

Bei der Auswahl des Systems muss beachtet werden, dass die Techdivision den Server für weitere AI Projekte verwenden will und daher ein System, welches den Chatbot und die Datenbank betreiben kann ohne ausgelastet zu sein benötigt wird.

Hierfür gibt es zwei Optionen, die zur Verfügung stehen.

1. Eine Compute Engine Instanz in der Google Cloud
2. Ein dedicated Hetzner Server

Der Hetzner Server der zur Auswahl stehen würde ist der aktuell einzige dedicated GPU Server, der bei Hetzner angeboten wird. Er hat einen Intel Core i5-13500 als CPU mit 6 Performance Cores und 8 Efficient Cores. 64 GB DDR4 RAM, 2x 1.92 TB Gen3 NVMe SSDs und eine Nvidia RTX 4000 SFF Ada Generation mit 20GB GDDR6 ECC. Die Kosten für diesen Server wären maximal, wenn der Server 730 Stunden im Monat läuft, 218.96€ + 94.01€ einmalige Setupgebühr.

Diese Hardware ist mehr als ausreichend für das System und ermöglicht ihn für mehr Applikationen als diesen Chatbot zu benutzen.

Wenn man einen ähnlichen Server in der Compute Engine von Google zusammenstellen würde, dann hat man aktuell die Möglichkeit GPUs hinzuzufügen in den Maschinen Kategorien Allzweck, Rechen optimiert und Beschleunigungsoptimiert. Für die Berechnung der Kosten wird erneut angenommen, dass der Server 730 Stunden im Monat läuft. Wenn wir einem N1 Allzweck Server 64GB RAM, 10 vCPUs, welche mindestens beötigt werden um 64 GB RAM auswählen zu könne, und eine Nvidia T4 GPU mit 16GB VRAM hinzufügen, befinden wir uns bei ca 650€. Augrund dessen wird der Hetzner Server, der bei voller Nutzung ca 430€ billiger im Monat ist bevorzugt.

4.8.5 Anbindung Google Chat

5 Entwicklung

Nach dem Abschluss der Konzeption kann zur Entwicklung über gegangen werden.

5.1 Vorbereitung

Bevor mit der Entwicklung begonnen werden kann, müssen ein paar Dinge vorbereitet werden, damit ohne Probleme entwickelt werden kann.

5.1.1 Modelle

Der erste Schritt ist es die Modelle vorzubereiten.

Hierfür werden die Modelle, die in 4.1 vorgestellt wurden über Huggingface gedownloadet. Dies ist relativ simple über Git LFS zu erledigen. Die Repositories der Modelle werden einfach über git clone kopiert. Wenn die Dateien fertig gedownloadet sind, kann man die Modelle für das Importieren in Ollama vorbereiten. Hierfür

5.1.2 Google Cloud

Hier geht es um die Vorbereitung des Google Cloud Projekts für die Verbindung mit der Google Chat API.

Der erste Schritt hierbei ist es, dein Google Cloud Projekt zu erstellen. Wenn dies erledigt ist, kann man die Google Chat API aktivieren.

5.1.3 Python Umgebung

5.2 System

Wie in der Konzeption beschrieben wird das System auf dem der Chatbot erstellt wird ist ein Hetzner Server, auf dem ebenfalls die Datenbank liegt

6 Vergleich

6.1 Evaluierung

Der finale Teil der Arbeit beschäftigt sich mit dem Vergleich der Performance, des Systems. Das System wurde mit vier verschiedenen LLM Modellen implementiert und wird im ersten Schritt mit dem Framework RAGAS (siehe 2.10) miteinander verglichen. Nach diesem Schritt werden die zwei bestabschneidensten Modelle für die finale Bewertung von Mitarbeitern aus der Techdivision miteinander verglichen.

6.2 RAGAS Vergleich

Die erste Ausscheidungsrunde wird über das Benchmarking Frameworks entschieden. Die vier LLM Modelle die hier zur Auswahl stehen sind finetuned von Metas Llama3, Mistrals Mistral 2 und Googles Gemma. Dazu kommt noch die originale Version von Microsofts Phi 3.

6.2.1 Testset

Der erste Schritt für den Vergleich ist es Testsets herzustellen anhand derer Ragas den Benchmark Test durchführen kann.

Die Variante die hierfür verwendet wird ist es einen TestsetGenerator per `TestsetGenerator.from_langchain()` herzustellen und mit diesem dann das Testset erstellen zu lassen.

Als Vorbereitung hierfür werden Daten benötigt. Die Daten die als Dokument geladen werden haben alle ein Metadata Dictionary. Damit der Generator erkennen kann, welche Daten zusammengehören brauchen die Dokumente in diesem Metadata Dictionary ein Key Value Pair mit dem Namen ”filename“. Der Wert des ”filename“ Schlüssels wird verwendet um bei der Generation der Testdaten zusammengehörige Dokumententeile zusammenzufügen.

Nach dem Vorbereiten der Daten kann man damit beginnen den Generator zu erstellen. Für diesen Generator werden drei Parameter verwendet. Die `generator_llm`, `critic_llm` und `embeddings`. Als Embedding Model benutzen wir ebenfalls das Model ïntfloat/multilingual-e5-large", wie für den Chatbot und den Importer. Als Critic und Generator LLM werden wir zwei verschiedene LLMs nutzen. Da die Geschwindigkeit der Erstellung dieser Daten keine große Rolle spielt werden hierfür große Modelle genutzt. Das Generator Modell wird eine auf Deutsch finetuned Version von Mistrals Mixtral 8x7B sein und zwar "VAGOsolutions/SauerkrautLM-Mixtral-8x7B"[**?**]. Als Critic Modell wird eine Version von Microsofts Phi 3 genutzt, welche ebenfalls auf Deutsch finetuned wurde. Die Version die genutzt wird ist "VAGOsolutions/SauerkrautLM-Phi-3-medium", welche eine finetuned Version von dem Modell ünsloth/Phi-3-medium-4k-instructïst.

Nach dem Erstellen des Generators und dem vor und aufbereiten der Daten kann man das Testset generieren lassen. Für die Generierung kann man zwei Parameter setzen. Man kann

bestimmen, wie viele Elemente im Set enthalten sein sollen. Dazu kann man die Verteilung der Fragen bestimmen. Die Fragen die durch den Generator erstellt werden können drei verschiedene Typen haben. Das Datenset, welches man als Ausgabe bekommt

6.3 Umfrage Vergleich

Hierfür wurden 5 Mitarbeiter aus dem Unternehmen mit einem Fragebogen ausgestattet und ihnen wurde Zugang zum System gegeben.

6.3.1 Fragebogen

Der Fragebogen wurde so erstellt, dass es keine genauen Fragen gibt, sondern nur Themengebiete, die abgefragt werden sollen. Das wichtige beim Auswählen der Themengebiete war, dass es Informationen waren, die auf der Confluence Instanz, auf die der Chatbot Zugriff hatte, verfügbar waren.

A Erstes Kapitel des Anhangs

Wenn Sie keinen Anhang benötigen, dann bitte einfach rausnehmen.